



Machine Learning and Pattern Recognition

Fingerprint Spoofing Detection

INAAM EL HELWE 306979

MARIA AKL 293829

August 18, 2024

Contents

1	Feature Analysis	1
2	Validation	4
2.1	Training Protocol	4
2.2	Gaussian Classifier	5
2.2.1	Expectations	5
2.2.2	Results and observations	5
2.2.3	Final Gaussian models conclusion	7
2.3	Logistic Regression	7
2.3.1	Linear Logistic Regression	7
2.3.2	Quadratic Logistic Regression	10
2.3.3	Final LR Conclusion	13
2.4	Support Vector Machine	13
2.4.1	Expectations	14
2.4.2	Linear SVM:	14
2.4.3	Polynomial Kernel SVM:	15
2.4.4	RBF Kernel SVM:	18
2.4.5	Final SVM conclusion:	19
2.5	Gaussian Mixture Model - GMM	19
2.5.1	Expectations	19
2.5.2	Result and Observations	19
2.5.3	Final GMM conclusion:	21
2.6	Conclusion for Validation	21
3	Calibration	22
4	Evaluation	23
4.1	Quadratic Logistic Regression	24
4.2	Quadratic Support Vector Machine	25
4.2.1	Polynomial Kernel	25
4.2.2	RBF Kernel	26
4.3	GMM	27
5	Conclusion	28

List of Figures

1	Raw features distribution of the fingerprints dataset	1
2	Cross features scatter plots	2
3	PCA scatter plot for training dataset (a) and LDA histogram for training dataset (b)	3
4	Heatmap for all dataset (a) and Explained Variance (b)	3
5	Heatmap for spoofed fingerprint (a) and authentic fingerprint (b)	4

6	minDCF values for different models with varying PCA	6
7	Logistic Regression with different PCA values for $\pi_T = \frac{1}{11}$	8
8	Logistic Regression with different PCA values for $\pi_T = 0.1$	9
9	Logistic Regression with different PCA values for $\pi_T = 0.5$	10
10	Quadratic Logistic Regression with different PCA for $\pi_T = \frac{1}{11}$	11
11	Quadratic Logistic Regression with different PCA for $\pi_T = 0.1$	12
12	Quadratic Logistic Regression with different PCA for $\pi_T = 0.5$	12
13	Quadratic Logistic Regression with different PCA for $\pi_T = 0.33$	12
14	Results of applying Linear SVM with PCA 6	14
15	Results of applying Linear SVM with PCA 8	15
16	Polynomial kernel of degree = 2 for K = 1 and c = 1 with z-norm	16
17	Polynomial kernel of degree = 2 and c = 1	16
18	Polynomial kernel of degree = 3 and c = 1 and K = 1	17
19	Results of applying RBF kernel with K = 1	18
20	non Target Model - Target Model	20
21	minDCF and DCF plots of the best models	22
22	minDCF and DCF plots of the best models with calibration	23
23	minDCF and DCF plots of best Quad LR	25
24	minDCF and DCF plots of best kernel RBF SVM	26
25	minDCF and DCF plots of best GMM model	28

The aim of this report is to create a classifier that can effectively differentiate between authentic and manipulated fingerprint images by leveraging various machine learning algorithms. These fingerprint images are initially transformed into a lower-dimensional space, typically reduced to just 10 dimensions to ensure computational efficiency. It's important to note that these reduced dimensions don't hold any specific physical meaning. The original dataset comprises 7 distinct classes. However, for the purposes of this project, we have simplified it by binarizing the dataset. We have grouped all manipulated (spoofed) fingerprint samples into class 0 and all genuine (authentic) fingerprint samples into class 1. In this project, we will conduct training on various classifiers and explore different hyperparameter combinations using our training dataset. This approach will allow us to assess the performance of the top-performing classifier on our test dataset and verify the suitability of our choice.

1 Feature Analysis

Our training dataset comprises 2,325 samples, with 1,525 belonging to class 0 and 800 to class 1. This results in an uneven distribution, with one class being twice as prevalent as the other. In the subsequent sections, we will use histograms to visually represent the distribution of various features before implementing any pre-processing techniques:

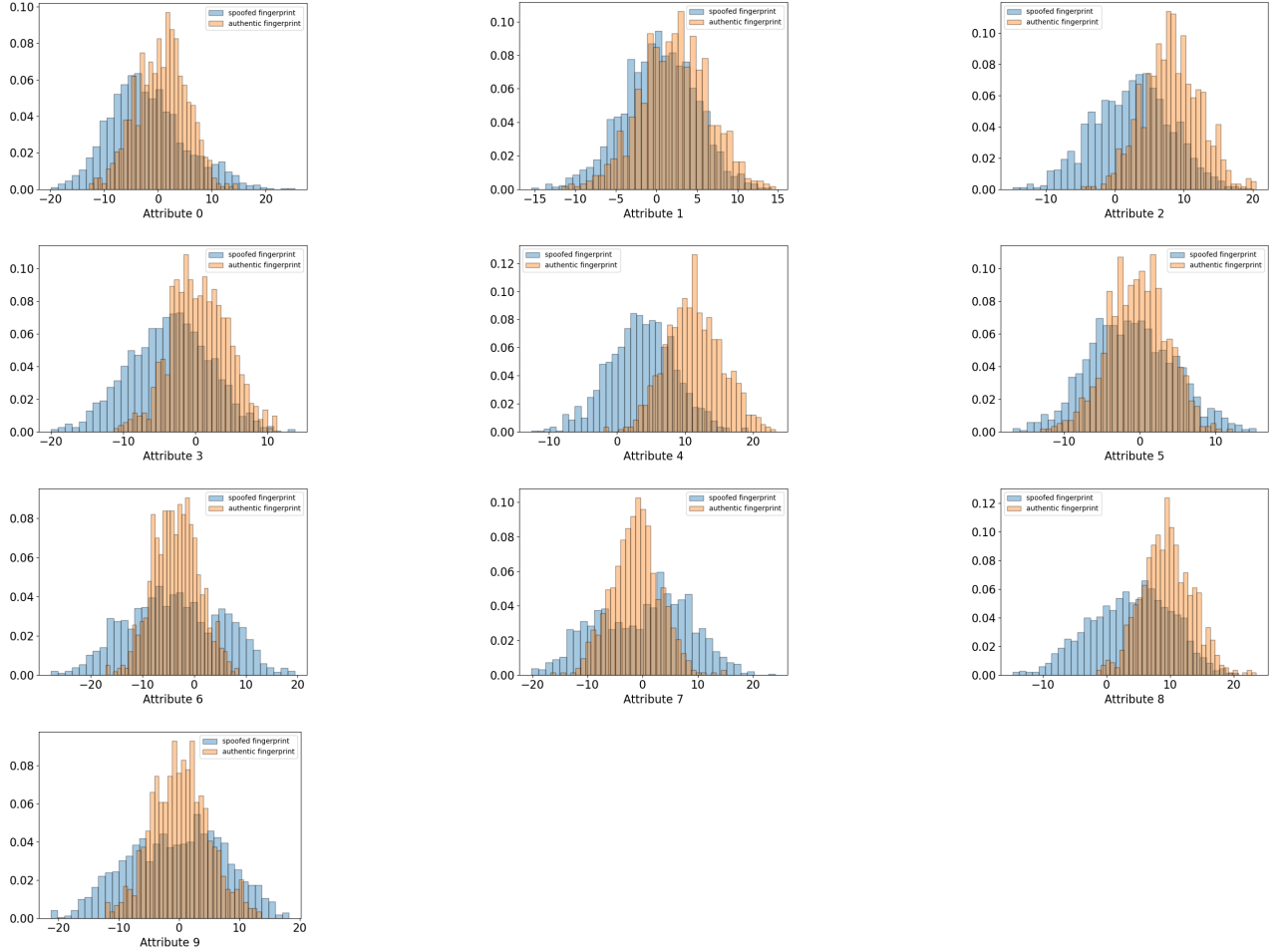


Figure 1: *Raw features distribution of the fingerprints dataset*

We observe that a Gaussian distribution effectively models the features of the target class, whereas Gaussian densities may not adequately capture the characteristics of the non-target class. It's important to note that, for certain features, the distributions provide a clear means of distinguishing between the two classes. For instance, attribute 4 emerges as the most discriminative feature in our case.

Next, we will visualize our dataset more effectively by presenting scatter plots of pairs of features and histograms for enhanced understanding:

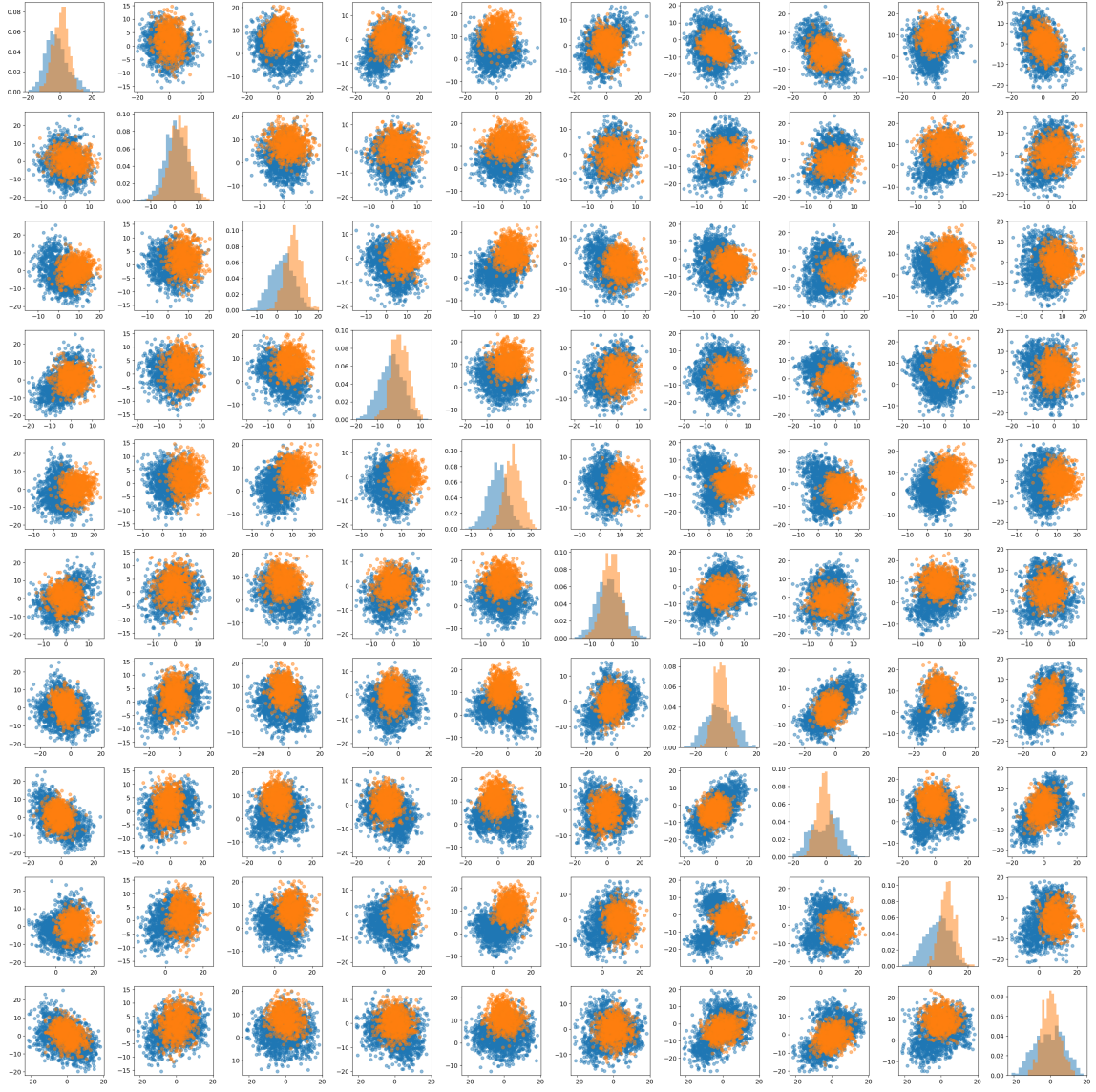


Figure 2: *Cross features scatter plots*

Analyzing the 2D scatter plots in Figure 2, it becomes evident that the target class shows a clear Gaussian density distribution. In contrast, for the non-target class, samples are spread across multiple clusters, as observed in plot(0,8) and plot(6,8).

We've applied two dimensional PCA and LDA transformations to gain a deeper understanding of our dataset. The results are illustrated in the following figures:

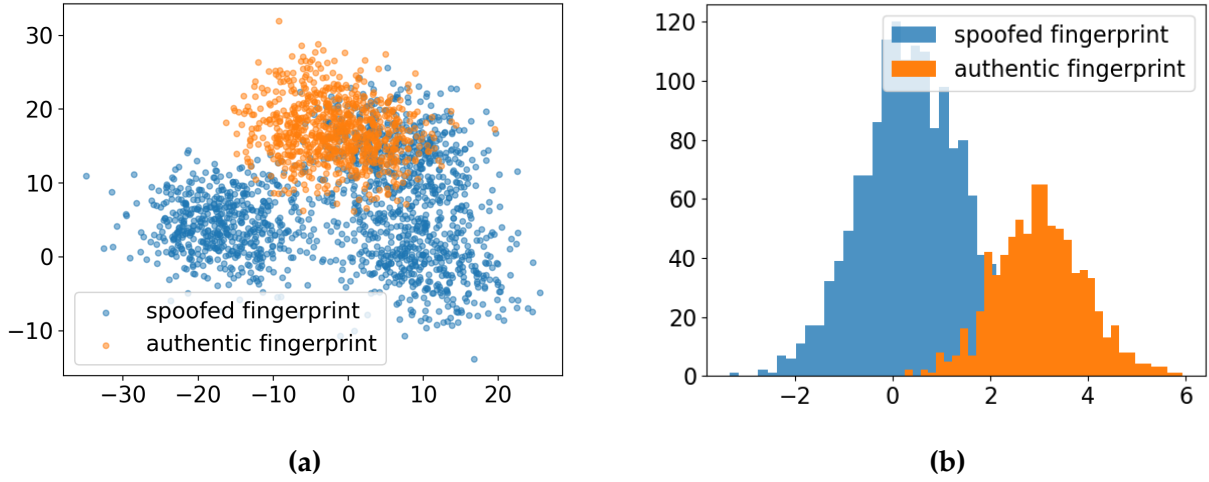


Figure 3: *PCA scatter plot for training dataset (a) and LDA histogram for training dataset (b)*

The PCA scatter plot reveals that the non-target label (spoofed fingerprints) shows multiple clusters, while the target class shows only one cluster. On the other hand, the LDA analysis indicates that the features are linearly separable, with a small region where distinguishing between the two classes becomes challenging and may lead to errors. Consequently, we anticipate that linear models like Logistic Regression may not perform poorly. However, considering the observed feature distribution in the scatter plot, we expect that non-Gaussian and/or non-linear models (e.g., Multivariate Gaussian) would be notably more effective for this task.

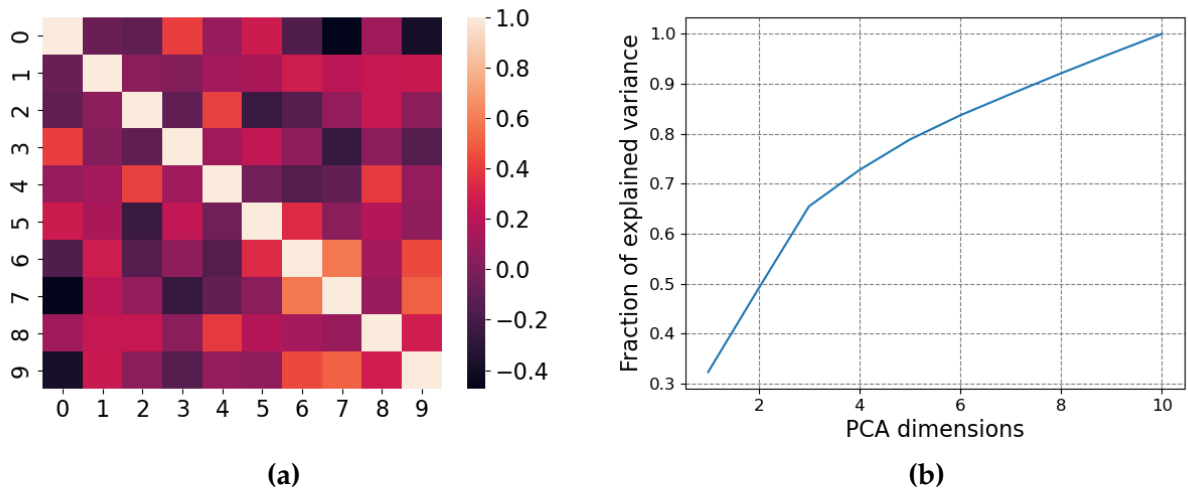


Figure 4: *Heatmap for all dataset (a) and Explained Variance (b)*

Features Correlation: Let's now explore the correlations among our features using Pearson correlation analysis. We begin by examining the heatmap of the entire dataset, as shown in Figure (a). It's evident that certain features demonstrate strong correlations, such as

attributes 6-7 and 7-9(subsequently 6-9), as well as 3-0. On the other hand, some features display weaker correlations, indicated by the darker colors in the heatmap. Given these observations, employing dimensionality reduction techniques like PCA becomes particularly valuable. This assumption finds strong support in Figure (b), where we observe that reducing our dimensions to 9, as opposed to the original 10, preserves 96% of the dataset’s variance. Even with just 8 dimensions, we can maintain approximately 92% of the variance.

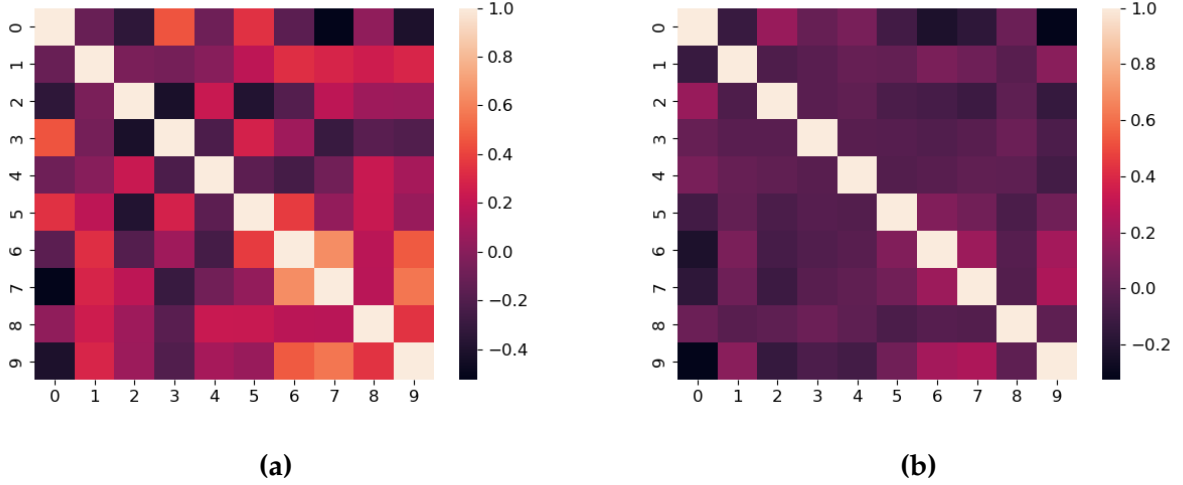


Figure 5: Heatmap for spoofed fingerprint (a) and authentic fingerprint (b)

Examining the Heatmap for each class separately: the case of authentic fingerprints, the features show relatively weak correlations. However, for the non-target class (spoofed fingerprints), there are noticeably stronger correlations. This suggests that employing PCA could potentially lead to the loss of valuable information that distinguishes the target class.

2 Validation

2.1 Training Protocol

In the upcoming section, we will assess various models and select the most appropriate one for our application. To conduct this analysis, we will employ a K-Fold protocol with $K=5$. This approach enhances the reliability of our model selection process since it provides larger amount of data for both training and validation purposes. We are using a seed value of 0 for our training, but it’s worth noting that results may vary significantly with different seed values. While cross-validation can help choose the best seed, it’s not within the scope of our project to cover this aspect.

It’s important to emphasize that we don’t apply any transformations to the entire training data before splitting it to avoid biasing the validation.

Our target application is characterized by the triplet ($\pi_T = 0.5, C_{fn} = 1, C_{fp} = 10$), where π_T represents the prior (which is unbiased in this case), and C_{fn} and C_{fp} denote the costs associated with false negatives and false positives, respectively. Our application’s working point is equivalent to another working point with uniform costs ($\tilde{\pi} = 1/11, C_{fn} = 1, C_{fp} = 1$). However, during our training, we will also consider one additional working point ($\tilde{\pi} =$

0.2, $C_{fn} = 1, C_{fp} = 1$) when training some classifiers for comparison purposes, in order to know how our optimal decision for each classifier would perform on a different target application.

In terms of model evaluation, our primary performance metric is minDCF which is a normalized metric that measures the cost we would incur if we made different decisions based on the recognizer scores.

2.2 Gaussian Classifier

Let's begin our analysis with the first set of classifiers, the Gaussian classifiers. These classifiers include various types:

1. Full Covariance (MVG): This classifier assumes that each class follows a Gaussian distribution with a full covariance matrix.
2. Diagonal Covariance (Naive Bayes): Here, it is assumed that features are not correlated, resulting in diagonal covariance matrices.
3. Tied MVG: In this case, each class maintains its unique mean, but they all share the same covariance, computed over the entire dataset.
4. Tied Naive Bayes: assumes the same diagonal covariance matrix for all classes.

Gaussian classifiers operate on the assumption that each class 'c' follows to a Gaussian distribution:

$$X|C \approx N(\mu_c, \Sigma_c)$$

2.2.1 Expectations

Analyzing the histograms, we notice that our data approximates Gaussian distributions. This suggests that the Multivariate Gaussian (MVG) classifier is likely to perform well. Additionally, examining the Pearson correlation matrices reveals few feature correlations, implying that Naive Bayes should not perform poorly, although it may slightly underperform compared to MVG.

Furthermore, our histograms indicate that data within each class exhibits varying spreads around the mean, resulting in distinct covariance matrices. Consequently, we anticipate that the MVG classifier with tied covariance may not perform as effectively as the other classifiers.

2.2.2 Results and observations

These plots display the results when we used different Gaussian classifiers with varying PCA dimensions:

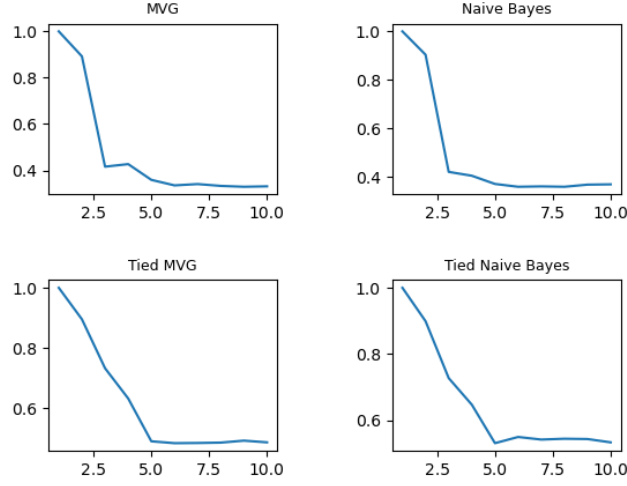


Figure 6: minDCF values for different models with varying PCA

Observing in Figure 6, we can see that PCA offers a slight enhancement in classifier performance. However, it's crucial to note that performance experiences a significant decline when reducing the feature space to fewer than 5 dimensions. Therefore, in the subsequent table, we'll present the numerical results of reducing dimensions to just 6, except for Tied Naive Bayes, where the optimal performance is observed with a PCA of dimension of 5:

Classifier	PCA	$\text{minDCF}(\tilde{\pi} = \frac{1}{11})$	$\text{minDCF}(\tilde{\pi} = 0.2)$
MVG	-	0.331	0.235
	9	0.330	0.239
	8	0.333	0.240
	7	0.341	0.238
	6	0.335	0.237
Naive Bayes	-	0.370	0.243
	9	0.369	0.247
	8	0.360	0.246
	7	0.361	0.248
	6	0.360	0.249
Tied MVG	-	0.486	0.367
	9	0.492	0.370
	8	0.485	0.370
	7	0.484	0.363
	6	0.483	0.360
Tied Naive Bayes	-	0.533	0.371
	9	0.543	0.374
	8	0.544	0.375
	7	0.541	0.376
	6	0.549	0.373
	5	0.530	0.389

Table 1: Gaussian Classifiers Results

Considering the results given for the application prior $\tilde{\pi} = \frac{1}{11}$, as expected, the MVG model delivers the best performance, while Tied MVG and Tied Naive Bayes show weaker results, aligning with our initial expectation that quadratic models outperform linear ones. We observe a small enhancement in results with PCA for the MVG model, but notably more substantial improvements emerge when applying PCA to the other models, particularly Naive Bayes. This improvement may be attributed to the removal of feature correlations. Regarding the alternative working point, it is evident that it produced better results. This can be attributed to the fact that our training set's empirical prior is 0.33, making 0.2 closer to this value than our target application prior.

2.2.3 Final Gaussian models conclusion

The best result has been obtained with the following configuration

$\tilde{\pi}$	PCA	Model	$\text{minDCF}(\tilde{\pi} = \frac{1}{11})$	$\text{minDCF}(\tilde{\pi} = 0.2)$
1/11	9	MVG	0.330	0.239

Table 2: Best configuration for Gaussian Models

2.3 Logistic Regression

2.3.1 Linear Logistic Regression

We shift now our attention to Logistic Regression models. As mentioned, the classes are unbalanced, so we will use a prior-weighted regularized version of the objective function:

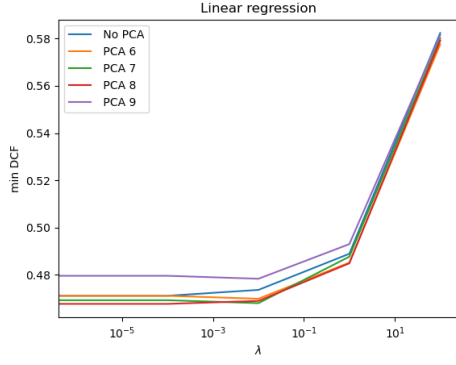
$$J(x, b) = \frac{\lambda}{2} ||w||^2 + \frac{\pi_L}{n_T} \sum_{i=1|c_i=1}^{n_T} \log(1 + e^{-z_i s_i}) + \frac{(1 - \pi_T)}{n_F} \sum_{i=1|c_i=0}^n \log(1 + e^{-z_i s_i})$$

with $s_i = (w^T x_i + b)$, where (w, b) are the model parameters, $\frac{\lambda}{2} ||w||^2$ a regularization term that helps obtaining a w with lower norm which reduces the risk of over-fitting and λ is a hyperparameter called regularization coefficient, when λ is too large, $||w||$ is small and we have poor separation (risk of under-fitting). However, when λ is small, $||w||$ is large and we have a good separation of classes for the training data but does not guarantee good generalization on unseen data.

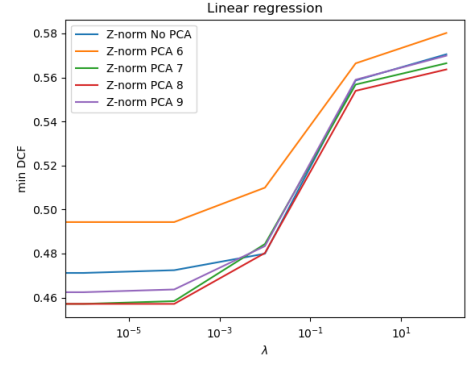
The choice of λ will be done through cross-validation so we're gonna plot minDCF graphs for a range of values of λ .

Expectation We expect this model to perform slightly worse than gaussian ones due to its assumption of a linear separation rule.

Results and observation In this model, we explore various combinations of the Prior parameter (π_T), the regularization coefficient (λ), and PCA as part of a comprehensive grid search approach. Therefore, we present these findings through visual plots to provide a more informative assessment of the performance of different models:



(a) Raw



(b) Z-normalized

Figure 7: Logistic Regression with different PCA values for $\pi_T = \frac{1}{11}$

λ	Raw	Z-norm
PCA = 6		
0	0.471	0.494
10^{-6}	0.471	0.494
10^{-4}	0.471	0.494
10^{-2}	0.467	0.510
1	0.485	0.566
100	0.578	0.580
PCA = 7		
0	0.469	0.457
10^{-6}	0.485	0.457
10^{-4}	0.469	0.457
10^{-2}	0.469	0.484
1	0.468	0.557
100	0.580	0.566
PCA = 8		
0	0.468	0.457
10^{-6}	0.468	0.457
10^{-4}	0.468	0.457
10^{-2}	0.469	0.480
1	0.485	0.554
100	0.579	0.564
PCA = 9		
0	0.480	0.462
10^{-6}	0.480	0.462
10^{-4}	0.480	0.464
10^{-2}	0.478	0.483
1	0.493	0.559
100	0.581	0.570
No PCA		
0	0.471	0.471
10^{-6}	0.471	0.471
10^{-4}	0.471	0.472
10^{-2}	0.473	0.480
1	0.489	0.560
100	0.582	0.571

Table 3: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for Linear Logistic Regression for $\pi_T = \frac{1}{11}$

Setting the prior to $\frac{1}{11}$, the highest performance achieved was 0.457 with PCA set to 8 or 7 when λ was less than or equal to 10^{-4} . However, since the best performance is consistently achieved with $\lambda = 10^{-4}$, we will consider it the optimal hyperparameter. This further confirms our previous hypothesis that linear separation surfaces perform worse than quadratic surfaces when it comes to our application.

Additionally, Z-normalization contributed in slight improvement in our model's performance. It's worth noting that very large values of λ consistently lead to worse results due to under-fitting.

Now retraining with different priors:

for $\pi_T = 0.1$ we get the following plots and can observe that similar patterns are being noted, with a slight improvement in performance:

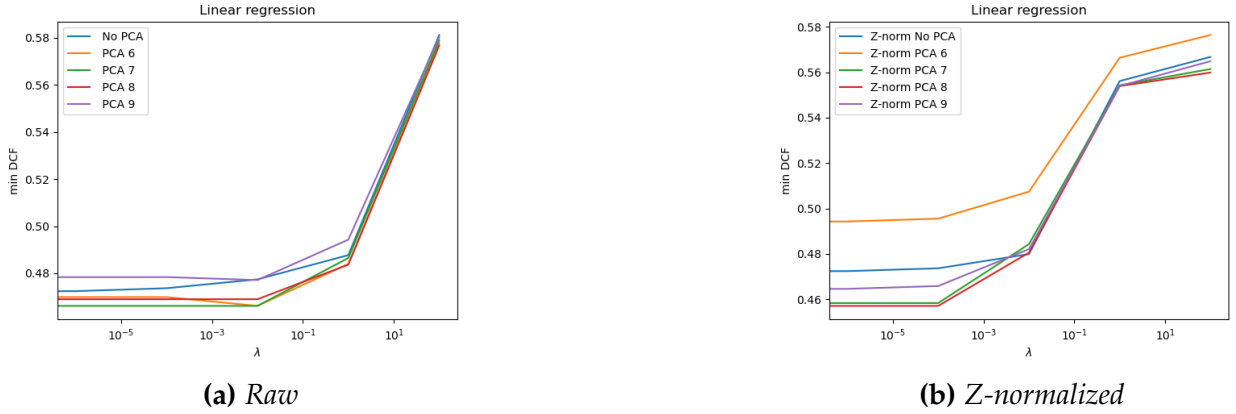
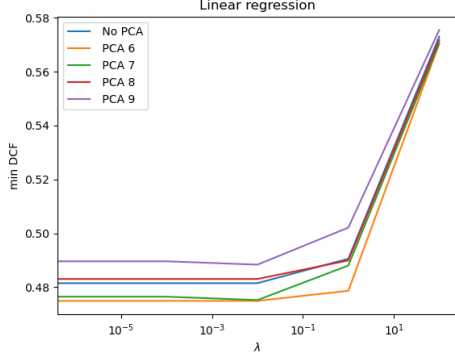


Figure 8: Logistic Regression with different PCA values for $\pi_T = 0.1$

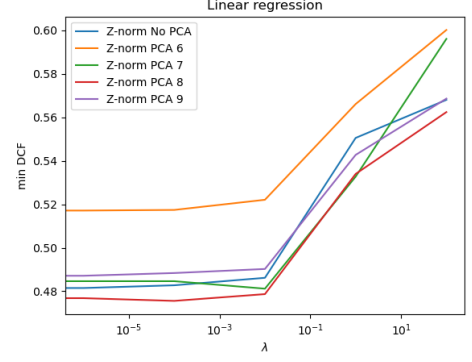
λ	Raw	Z-norm
PCA = 6		
10^{-4}	0.467	0.496
10^{-2}	0.466	0.507
PCA = 7		
10^{-4}	0.466	0.457
10^{-2}	0.466	0.484
PCA = 8		
10^{-4}	0.469	0.457
10^{-2}	0.481	0.474

Table 4: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for Quadratic Logistic Regression for $\pi_T = 0.1$

For $\pi_T = 0.5$ we obtain:



(a) Raw



(b) Z-normalized

Figure 9: Logistic Regression with different PCA values for $\pi_T = 0.5$

λ	Raw	Z-norm
PCA = 6		
10^{-4}	0.475	0.517
10^{-2}	0.475	0.522
PCA = 7		
10^{-4}	0.476	0.485
10^{-2}	0.475	0.481

Table 5: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for Quadratic Logistic Regression for $\pi_T = 0.5$

So we can observe that setting π_T to 0.1 lead to the best result which can be explained by the fact that it's biased towards the samples belonging to the non-target class which matches our unbalanced dataset.

2.3.2 Quadratic Logistic Regression

Let us now consider quadratic logistic regression, its a model that enables us to estimate the probabilities of belonging to a binary class as a function of continuous or categorical explanatory variables. It also includes quadratic terms of continuous variables, allowing us to capture any non-linear effects of explanatory variables on the response variable.

Another crucial aspect of quadratic logistic regression is the transformation of explanatory variables. We can utilize the following function to create an expanded feature space that incorporates the outer product of the x vector with itself and the original x vector:

$$\phi(x) = \begin{pmatrix} \text{vec}(xx^T) \\ x \end{pmatrix}$$

By using $\phi(x)$ feature vectors instead of x , we can train the logistic regression model and calculate linear separation rules for $\phi(x)$. This corresponds to estimating quadratic separation surfaces in the original space.

Expectations Until now the best results were obtained with MVG (which is a quadratic model),so we expect that quadratic LR perform better than linear version, because our data seems to be better separated by quadratic rules.

Results and observations Following the same approach for the quadratic version we will train with same set of priors and since we don't except $\pi_T = 0.5$ to improve the performance of our models we're going to check the results when embedding $\pi_T = 0.33$ that matches our application's empirical prior as well :

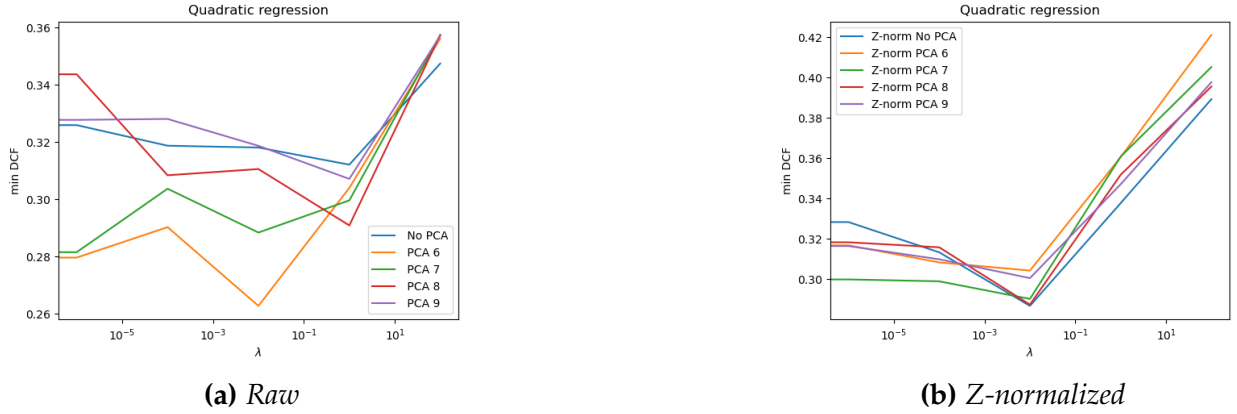


Figure 10: Quadratic Logistic Regression with different PCA for $\pi_T = \frac{1}{11}$

λ	Raw	Z-norm
PCA = 6		
10^{-4}	0.290	0.308
10^{-2}	0.262	0.308
PCA = 7		
10^{-4}	0.304	0.299
10^{-2}	0.288	0.290
PCA = 8		
10^{-4}	0.308	0.316
10^{-2}	0.311	0.287
PCA = 9		
10^{-4}	0.328	0.310
10^{-2}	0.319	0.301
No PCA		
10^{-4}	0.319	0.313
10^{-2}	0.318	0.287

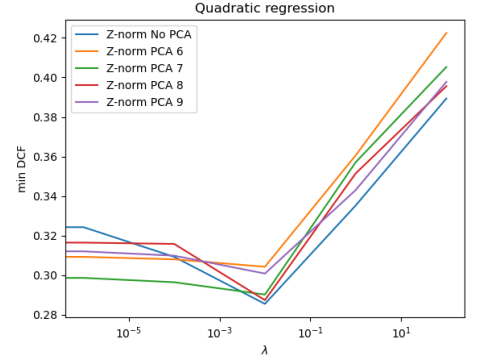
Table 6: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for Quadratic Logistic Regression for $\pi_T = \frac{1}{11}$

Based on the results presented above, we can infer that the use of quadratic logistic regression, along with $\pi_T = \frac{1}{11}$, showed enhanced performance for our working point. In fact, it outperformed the Gaussian models. This improved performance can be attributed to

our earlier observation that the non-target class does not conform perfectly to a Gaussian distribution. The most optimal configuration was achieved without normalization, and by setting the PCA dimension to 6, and using a lambda value of 10^{-2} .

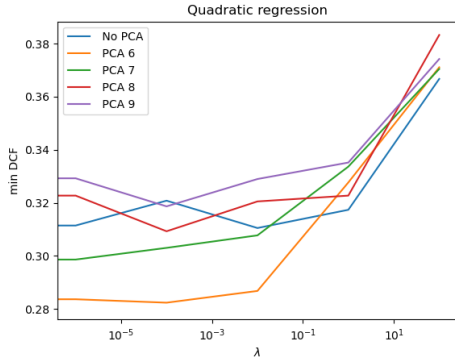


(a) Raw

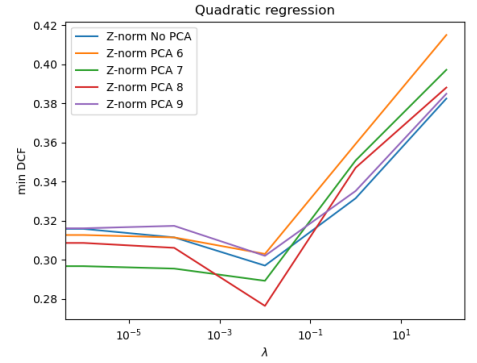


(b) Z-normalized

Figure 11: Quadratic Logistic Regression with different PCA for $\pi_T = 0.1$

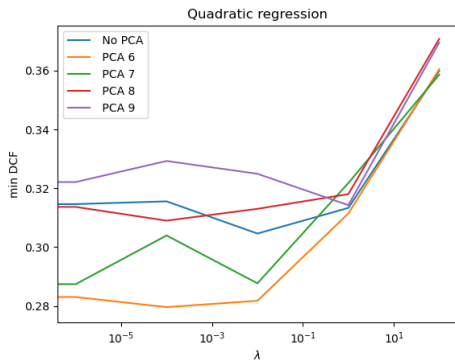


(a) Raw

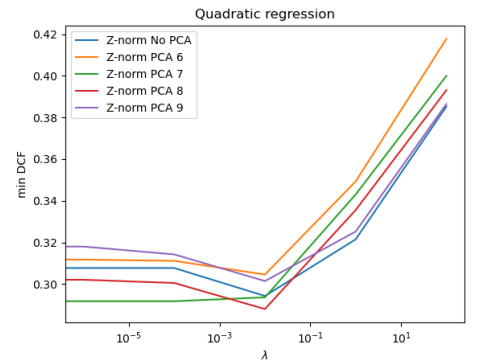


(b) Z-normalized

Figure 12: Quadratic Logistic Regression with different PCA for $\pi_T = 0.5$



(a) Raw



(b) Z-normalized

Figure 13: Quadratic Logistic Regression with different PCA for $\pi_T = 0.33$

The best results obtained for applying different priors π_T are reported in the tables below:

PCA = 6		
λ	Raw	Z-norm
10^{-2}	0.266	False

Table 7: Best configuration for $\pi_T = 0.1$

PCA = 8		
λ	Raw	Z-norm
10^{-2}	0.276	True

Table 8: Best configuration for $\pi_T = 0.5$

PCA = 6		
λ	Raw	Z-norm
10^{-4}	0.279	False

Table 9: Best configuration for $\pi_T = 0.33$

From the above results, we can see that $\pi_T = 0.1$ performed well while performance decreased for $\pi_T = 0.5$ and surprisingly also decreased with $\pi_T = \text{empiricalprior} = 0.33$, so we can conclude that a matched prior does not always give the best result. While the best configuration is still given by setting π_T to $\frac{1}{11}$ and applying a PCA = 6, $\lambda = 10^{-2}$ and without Z-norm.

2.3.3 Final LR Conclusion

The best result was obtained with the following configuration:

π_T	PCA	λ	Z - norm	minDCF($\tilde{\pi} = \frac{1}{11}$)	minDCF($\tilde{\pi} = 0.2$)
1/11	6	10^{-2}	False	0.262	0.1897

Table 10: Best configuration for Logistic Regression

Training our best obtained LR model with the working point having $\tilde{\pi} = 0.2$ gave better results same as with Gaussian models for same reason mentioned before.

2.4 Support Vector Machine

SVM models are discriminant non probabilistic models having output scores with no probabilistic interpretation. SVM tries to find the optimal hyperplane to divide the classes by maximizing the between classes margin. To solve the SVM problem, we use the dual formulation which is easier to optimize. The objective function that we want to maximize will be:

$$\max_{\alpha} \alpha^T 1 - \frac{1}{2} \alpha^T H \alpha$$

subject to

$$0 \leq \alpha_i \leq C_i \quad i = 1, \dots, n \quad \text{and} \quad \sum_{i=1}^n \alpha_i z_i = 0$$

The dual SVM formulation depends on the samples through dot products

$$H_{ij} = z_i z_j x_i^T x_j$$

Which allows us to have quadratic hyperplanes without explicitly expanding the feature space, We only need a kernel function that computes the dot product in the expanded space $k(x_i, x_j) = \phi(x_i)^T \phi(x_j)$.

We will exploit three versions of SVM

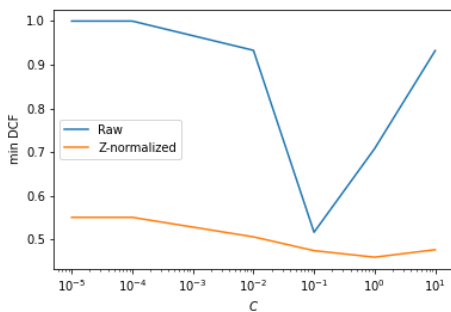
- Linear SVM
- Quadratic SVM
 - Polynomial kernel of degree d using the kernel: $k(x_1, x_2) = (x_1^T x_2 + c)^d$
 - Radial Basis Function kernel (RBF): $k(x_1, x_2) = e^{-\gamma \|x_1 - x_2\|^2}$

2.4.1 Expectations

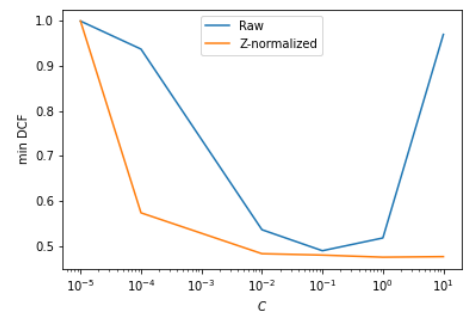
For Linear SVM we expect similar performance as the one given by Linear Logistic Regression, meanwhile the Quadratic separation surfaces should give better results.

2.4.2 Linear SVM:

We take a quick look at linear SVM models, even though we expect the results to perform poorly. We analyze the behavior of the model for PCA equal to 6 and 8 since these 2 values performed the best in our previous models, we fix K=1 to and we observe the results after rebalancing the classes according to our target application and when we don't consider rebalancing :

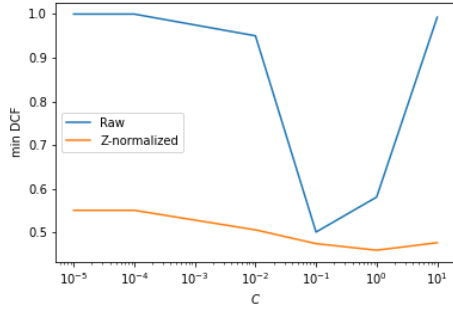


(a) PCA 6 Rebalanced

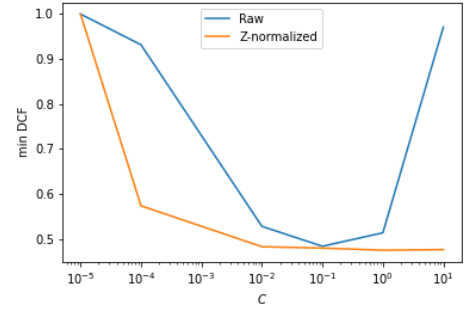


(b) PCA 6 with No rebalancing

Figure 14: Results of applying Linear SVM with PCA 6



(a) PCA 8 Rebalanced



(b) PCA 8 with No rebalancing

Figure 15: Results of applying Linear SVM with PCA 8

C	Rebalanced	Raw	Z-norm
PCA = 6			
10^{-5}	True	1	0.551
10^{-4}	True	0.933	0.506
10^{-2}	True	0.517	0.475
1	True	0.708	0.460
10	True	0.932	0.477
10^{-5}	False	1	0.573
10^{-4}	False	0.937	0.483
10^{-2}	False	0.536	0.480
1	False	0.490	0.475
10	False	0.970	0.476
PCA = 8			
10^{-5}	True	0.1	0.551
10^{-4}	True	0.950	0.506
10^{-2}	True	0.501	0.475
1	True	0.581	0.460
10	True	0.992	0.477
10^{-5}	False	1	0.573
10^{-4}	False	0.931	0.483
10^{-2}	False	0.528	0.480
1	False	0.483	0.475
10	False	0.971	0.476

Table 11: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for Linear SVM

As expected, we have poor results for our task using linear SVM, similarly to what we saw using linear logistic regression.

2.4.3 Polynomial Kernel SVM:

Since PCA 6 proved better performance in Linear SVM as well as in Quadratic Logistic Regression we will only train our following models with PCA = 6 and with no PCA , due to

the fact that training polynomial SVM is computationally expensive. First we will set the degree of the polynomial kernel to 2 and we fix c to 1

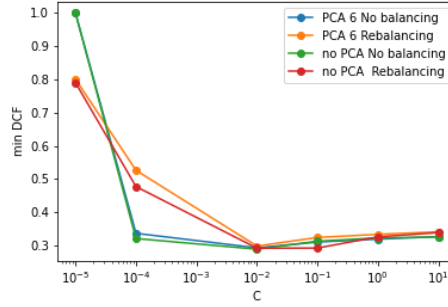


Figure 16: Polynomial kernel of degree = 2 for $K = 1$ and $c = 1$ with z -norm

C	Rebalanced	Raw	Z-norm
No PCA			
10^{-2}	False	0.288	True
PCA = 6			
10^{-2}	False	0.292	True

Table 12: Best $\min DCF(\tilde{\pi} = \frac{1}{11})$ reported in the above figure

We can see how results were significantly better than applying Linear SVM, specially for the cases with no rebalancing for both models trained with PCA = 6 and with no PCA for $C = 10^{-2}$.

Eventhough performance with no PCA is slightly better it's more computationally expensive than not applying PCA so we will consider the model given by applying PCA with dimension = 6 as the best model for SVM so far.

We can conclude that rebalancing does not seem effective for SVM in this task so we will stop considering it in the following trainings of degree 2. Now tuning the hyper-parameter K and checking if z -norm is also beneficial in case of Polynomial SVM with degree 2, we obtain the following graph:

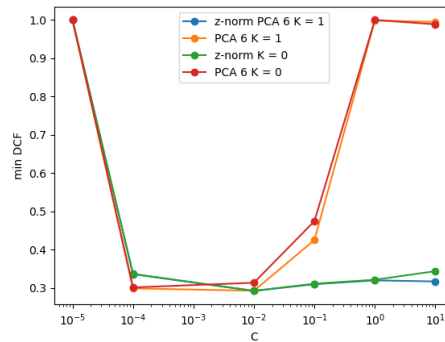


Figure 17: Polynomial kernel of degree = 2 and $c = 1$

We can conclude that $K = 1$ performs slightly better than $K = 0$ and z-normalization proved to give better performance. Best configuration for 2^{nd} degree polynomial is reported in the below table. Noting once again that we discarded the case of No PCA because of its computational inefficiency and applying PCA allows us to work with less features and with almost as good results:

C	Rebalanced	K	Z-norm	minDCF($\tilde{\pi}$)
PCA = 6				
10^{-2}	False	1	True	0.292

Table 13: Best configuration for 2^{nd} degree polynomial

We try now with degree $d = 3$ fixing $K = 1$:

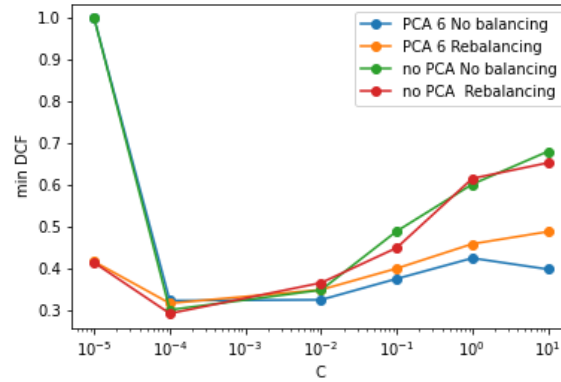


Figure 18: Polynomial kernel of degree = 3 and $c = 1$ and $K = 1$

In case of $d = 3$ we're going to report values corresponding to $C = 10^{-4}$ and $C = 10^{-2}$ since it can be seen from the figure that they give the best results:

C	Rebalanced	Raw	Z-norm
No PCA			
10^{-4}	True	0.294	True
10^{-2}	True	0.367	True
10^{-4}	False	0.350	True
10^{-2}	False	0.491	True
PCA = 6			
10^{-4}	True	0.318	True
10^{-2}	True	0.351	True
10^{-4}	False	0.327	True
10^{-2}	False	0.377	True

Table 14: Best $\min\text{DCF}(\tilde{\pi} = \frac{1}{11})$ reported in the above figure

In case of PCA = 6 performance dropped significantly compared to applying polynomial kernel of 2^{nd} degree, this can be explained by the fact that when increasing the degree

we increase the risk of overfitting aswell. We can note that rebalancing classes gave better results in case of $d = 3$. Trainig with no PCA in case of 3^{rd} degree polynomial gave better results than PCA = 6 but still worse than applying 2^{nd} degree Polynomial. Since increasing the degree did not prove any significant improvement to our application we're gonna stop at degree 3.

2.4.4 RBF Kernel SVM:

Analyzing now the behaviour of the model using RBF Kernels, based on previous results we're not going to rebalance our models and we will fix $K = 1$ and apply z-normalization and we will perform fine tuning for different values of γ to find the optimal one :

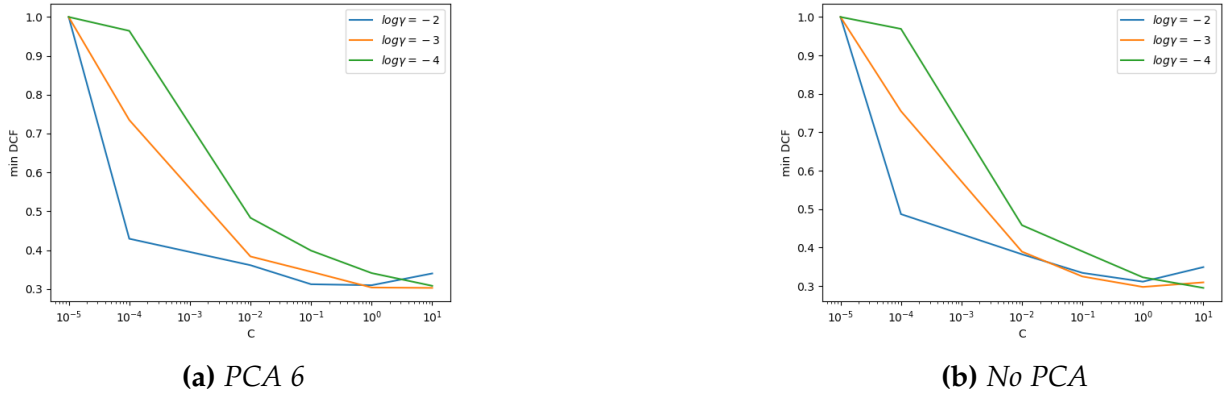


Figure 19: Results of applying RBF kernel with $K = 1$

	$C = 10^{-1}$	$C = 10^0$	$C = 10^1$
$\log \gamma = -2$	0.312	0.309	0.339
$\log \gamma = -3$	0.344	0.304	0.303
$\log \gamma = -4$	0.399	0.340	0.308

Table 15: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for RBF kernel results for PCA = 6

	$C = 10^{-1}$	$C = 10^0$	$C = 10^1$
$\log \gamma = -2$	0.334	0.311	0.349
$\log \gamma = -3$	0.325	0.297	0.310
$\log \gamma = -4$	0.390	0.323	0.295

Table 16: $\min DCF(\tilde{\pi} = \frac{1}{11})$ for RBF kernel results for no PCA

We observe that applying RBF Kernel did not improve performance, but it's worth noting that best values were obtained by setting γ to 10^{-3} . Also for RBF kernel PCA did provide worse results but close enough to not applying PCA so it's better to consider the model with PCA = 6 as the best one for this kernel since it allows us working with less features.

2.4.5 Final SVM conclusion:

The best configuration for SVM is giving by applying Quadratic SVM with polynomial kernel function of degree 2 :

C	Rebalanced	K	Z-norm	minDCF($\tilde{\pi}=\frac{1}{11}$)	minDCF($\tilde{\pi}=0.2$)
PCA = 6					
10^{-2}	False	1	True	0.292	0.201

Table 17: Best configuration of SVM for $\tilde{\pi} = \frac{1}{11}$ and $\tilde{\pi} = 0.2$

2.5 Gaussian Mixture Model - GMM

Gaussian mixtures are the last type of classifier for this task, this model finds utility in density estimation tasks, operating on the assumption that our data can assume distribution across multiple Gaussian components, often referred to as clusters. In the subsequent analysis, we will explore all permutations of the Full-Covariance model, the Diagonal model, the Tied diagonal and Tied Full-Covariance model for both target and non-target classes, exploring as well different number of clusters for each class.

2.5.1 Expectations

As mentioned in section 2 the target class is expected to give good results since it seems well described by a Gaussian density. While non-target class is composed of different sub-classes but each sub-class seem to follow a Gaussian density as well. So since it is possible to choose different number of components for GMM of different classes, different combinations have been tested. We expect that a small number of components for target class and a large number of components for non-target class provide good performances. So in this case the hyperparameter to tune is the number of components of each Gaussian we will consider 1,2,4 components for the target class and 2,4,8,16 components for the non-target class, taking into consideration that increasing the number of components increases our risk of over-fitting. All our trainings will cover the data without z-normalization.

2.5.2 Result and Observations

We're going report only the best results given by testing many different combinations of:

- number of GMM components for both target and non-target classes
- GMM types Full, Diagonal and Tied

We're going to report plots of some of the best results of trying the different combinations of models and number of components for PCA=6 that proved good performance in the previous trainings and no PCA:

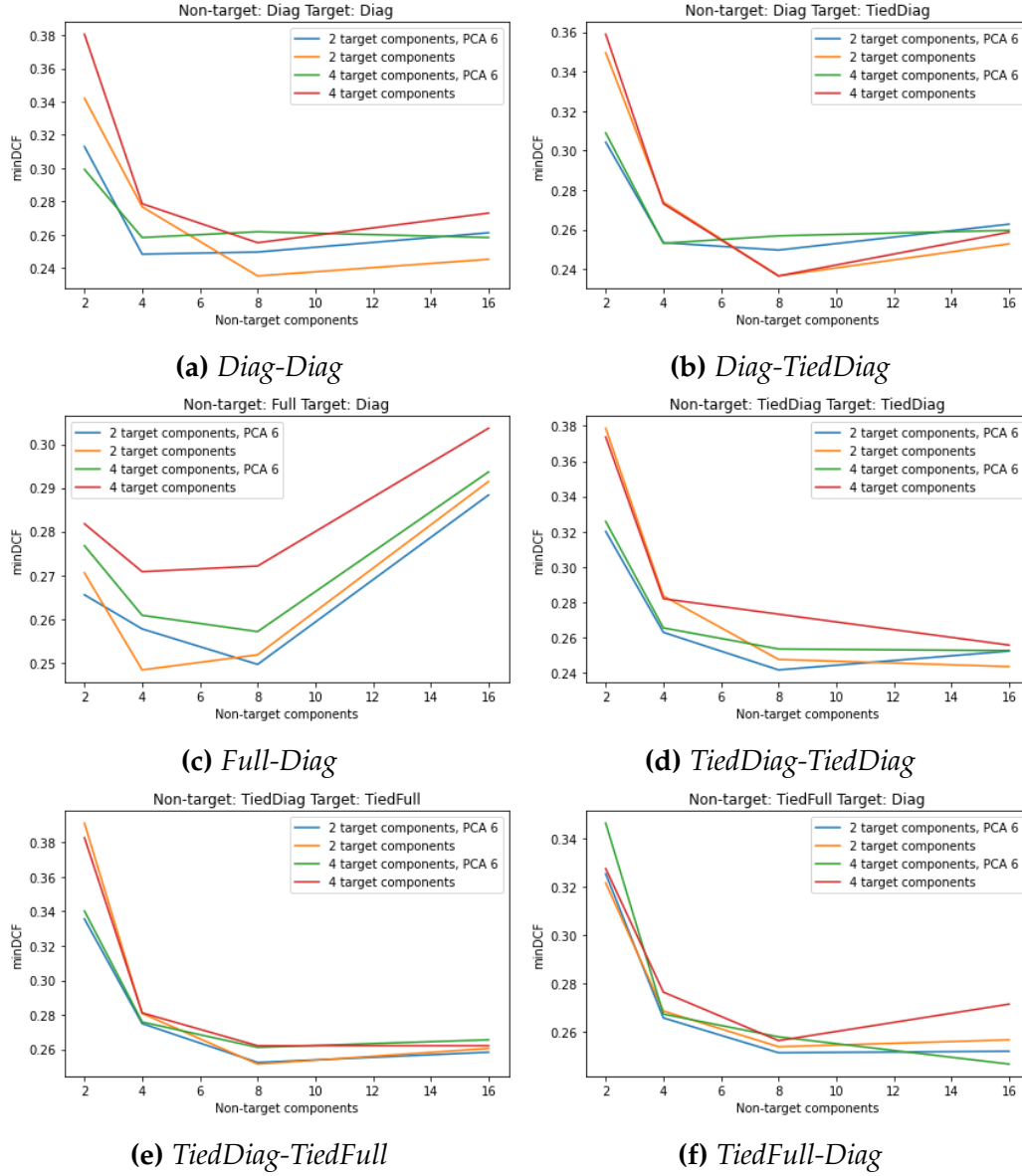


Figure 20: non Target Model - Target Model

modelTarget	Non-Target,Target	PCA 6	Non-Target,Target	no PCA
Full	(8,2)	0.252	(16,2)	0.253
Diag	(8,2)	0.243	(8,1)	0.242
TiedDiag	(8,2)	0.241	(16,2)	0.243
Tied	(8,2)	0.252	(8,2)	0.251

Table 18: Best results of tiedDiag(nonTarget)

modelTarget	Non-Target,Target	PCA 6	Non-Target,Target	no PCA
Full	(4,2)	0.249	(8,2)	0.259
Diag	(4,2)	0.248	(8,2)	0.235
TiedDiag	(8,2)	0.249	(8,2)	0.236
TiedFull	(4,2)	0.258	(8,2)	0.252

Table 19: Best results of Diag(nonTarget)

modelTarget	Non-Target,Target	PCA 6	Non-Target,Target	no PCA
Full	(4,2)	0.255	(4,2)	0.260
Diag	(8,2)	0.249	(4,2)	0.248
TiedDiag	(4,2)	0.250	(4,2)	0.249
TiedFull	(4,2)	0.256	(4,2)	0.252

Table 20: Best results of Full(nonTarget)

modelTarget	Non-Target,Target	PCA 6	Non-Target,Target	no PCA
Full	(16,2)	0.251	(4,2)	0.251
Diag	(16,4)	0.246	(8,2)	0.253
TiedDiag	(16,4)	0.248	(8,4)	0.257
TiedFull	(16,2)	0.257	(4,4)	0.255

Table 21: Best results of TiedFull(nonTarget)

As anticipated, the best performances are always reported with number of components equal to 4 and 8 (specially 8) for the non-target class which can be explained by our previous knowledge that the non-target class is constituted of 6 sub-classes, while for the target class setting the number of components to 2 was ideal. Applying Diagonal and Tied Diagonal covariance for both spoofed and authentic classes led to the best performances so we can conclude that both classes are constituted of similarly distributed clusters and with poor correlations between features for each class.

Models trained without applying PCA performed better.

2.5.3 Final GMM conclusion:

Since we finished training we can conclude that applying GMM with number of components equal to 8 and 2 for non-target class and target class respectively, with no PCA provided the best performance on our dataset:

Non-Target,Target	Non-Target,Target Model	PCA	minDCF($\tilde{\pi} = \frac{1}{11}$)	minDCF($\tilde{\pi} = 0.2$)
(8,2)	Diag-Diag	No	0.235	0.168

Table 22: Best GMM for $\tilde{\pi} = \frac{1}{11}$ and $\tilde{\pi} = 0.2$

2.6 Conclusion for Validation

After training many models, the best models are three, their configurations are reported in the below table:

Model's configuration	minDCF($\tilde{\pi} = 1/11$)
GMM $G_{target}=2$, $G_{nontarget}=8$, $Model_{target}=Diag$ $Model_{nontarget} = Diag$, $PCA=0$ $Znorm=False$	0.235
Quadratic Logistic regression $PCA = 6$, $\pi_T = \frac{1}{11}$, $\lambda = 10^{-2}$, $Znorm = False$	0.262
PolyKernel SVM $PCA = 6$, $d = 2$, $C = 10^{-2}$, $Rebalanced = False$, $K = 1$, $c = 1$, $Znorm = True$	0.292

Table 23: Best Models

In general our expectations were validated and the best performing models are non-linear. Quadratic logistic regression and polynomial SVM with degree 2 both introduce non-linear decision boundaries which are convenient for our data. While GMM makes it possible to handle the 2 classes differently which is important since each class showed a different distribution and different number of components, GMM yields globally the best performance.

3 Calibration

We selected the most promising models based on their minDCF which depends on a threshold used to perform class assignments. In the following we want to observe the performance of the best selected models using this threshold and the theoretical threshold. The analysis we conduct refers to the detection cost function using the optimal theoretical threshold, the metric is referred to as actual DCF (actDCF). If scores are poorly calibrated, we choose a transformation function that performs the mapping $s \rightarrow s_{cal} = f(s)$

$$f(s) = \alpha s + \beta - \log \frac{\tilde{\pi}}{1 - \tilde{\pi}} \quad (1)$$

α and β are estimated by a Linear Logistic Regression. we recover the calibrated score by subtracting the theoretical threshold. To estimate the parameters of the calibration function we will use cross validation.

We expect GMM and Quadratic LR to produce well calibrated scores whereas SVM no, since it gives score with no probabilistic interpretation.

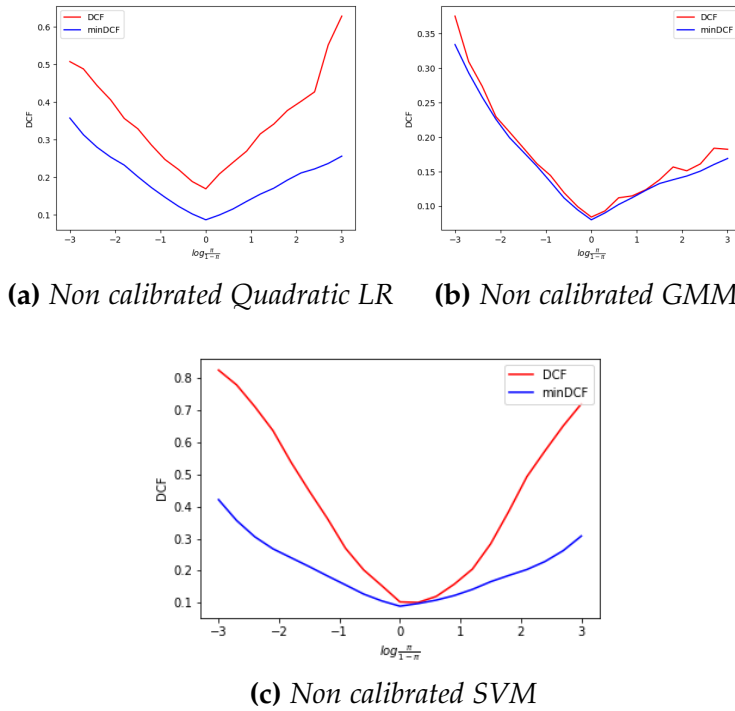


Figure 21: minDCF and DCF plots of the best models

We can observe that GMM produced very well calibrated scores while SVM and LR's scores are not calibrated. We apply calibration on all models:

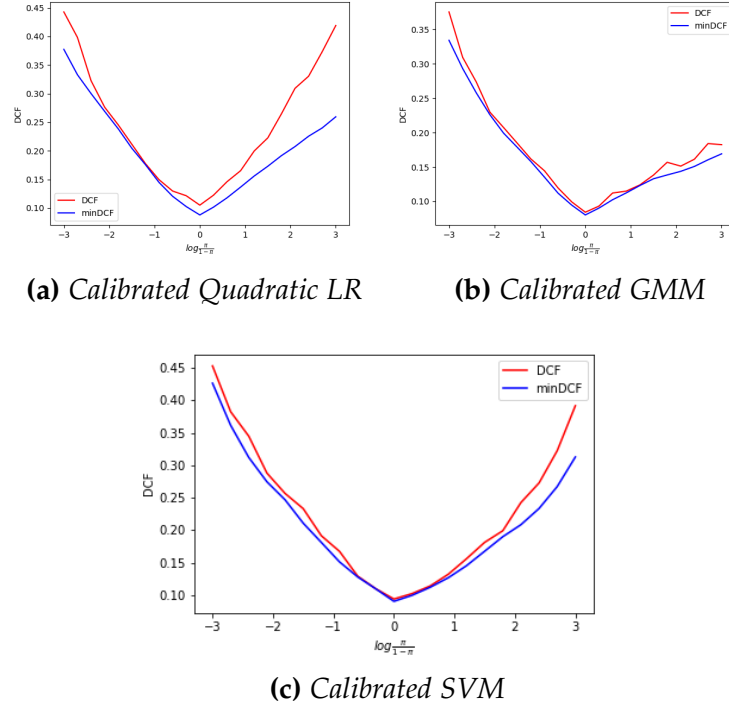


Figure 22: *minDCF and DCF plots of the best models with calibration*

Considering quadratic logistic regression model, we can still notice some calibration issues at certain working points. However, when it comes to the Support Vector Machine (SVM), there was a notable improvement in results after calibration.

4 Evaluation

The last step of our task is now to analyze the performance of our models on the evaluation set, and check if the choices we made were optimal. Models are now trained on the whole training set (with no cross validation) and tested on test set. We will check the performance of our three best models in the validation phase but we will also consider combinations of different hyper-parameters and feature space dimensions to check if models that we discarded would perform better on the test set. In the following, we will discard linear models due to their bad performance. We will not consider gaussian models as they are already integrated in GMM and due to their really poor performance compared to other applied classifiers.

First reporting the performance of our optimal classifiers:

Model	Eval minDCF($\tilde{\pi} = 1/11$)	Val minDCF($\tilde{\pi} = 1/11$)	Eval minDCF($\tilde{\pi} = 0.2$)
GMM $G_t=2, G_{nt}=8$ Diag, Diag	0.218	0.235	0.139
Quadratic LR PCA = 6	0.251	0.262	0.160
PolyKernel SVM PCA = 6 – $d = 2$	0.269	0.292	0.188

Table 24: Best Models with $\tilde{\pi} = \frac{1}{11}$ and $\tilde{\pi} = 0.2$

As we can see, minDCF values are smaller in the evaluation phase with respect to the values obtained in the validation phase. We can say that the chosen classifiers performed better than we expected on our test data. Now we will check minDCF for other discarded models.

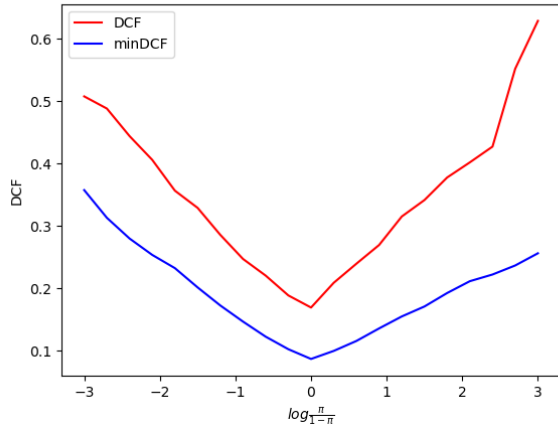
4.1 Quadratic Logistic Regression

We will consider our best LR model for the same λ interval that we used during the training phase and fixing π_T to $\frac{1}{11}$, we will take into account only the case of non z-normalized data with 6 and 8 principle components. We will highlight the top-performing model during the evaluation phase in red and showcase the model that performed the best during the validation phase in blue.

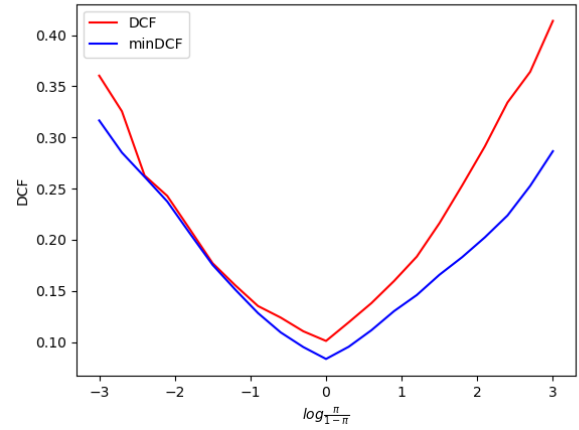
λ	evalPCA = 6	evalPCA = 8
10^{-6}	0.251	0.270
10^{-4}	0.252	0.269
10^{-2}	0.251	0.273
0	0.247	0.277
1	0.257	0.266
100	0.293	0.290

Table 25: comparing performance of Quadratic LR on validation and evaluation data

Our optimal model in validation phase did well on the test data, but it wasn't the optimal choice. Training with a regularization parameter of 0 yielded slightly better results compared to $\lambda = 10^{-2}$, but our selection of Principal Component Analysis (PCA) proved to be a good decision. Adding calibration, we can see a slight improvement.



(a) Non calibrated Quadratic LR



(b) Calibrated Quadratic LR

Figure 23: *minDCF and DCF plots of best Quad LR*

4.2 Quadratic Support Vector Machine

As mentioned , we will not consider the linear version of the model, lets compare PolyKernel and RBFKernel SVM with various hyperparameters:

4.2.1 Polynomial Kernel

In the following we will consider versions with Rebalanced=False and Znornalization=True. We will consider our best model that is Polynomial kernel SVM d=2 with PCA=6. We will evaluate the same model without PCA , and we will consider different values of C within the same interval that we used for training and tune the hyperparameter K between 0 and 1. Results are reported in the following tables:

C	PCA=None	PCA=6
10^{-4}	0.262	0.293
10^{-2}	0.252	0.269
10^{-1}	0.257	0.277
1	0.275	0.277
10	0.284	0.274

Table 26: *Polynomial SVM d=2 ,K=1, Znorn=True, Rebal=False*

C	PCA=None	PCA=6
10^{-4}	0.262	0.294
10^{-2}	0.251	0.274
10^{-1}	0.257	0.278
1	0.276	0.278
10	0.282	0.277

Table 27: *Polynomial SVM d=2 ,K=0, Znorn=True, Rebal=False*

Not applying PCA for polynomial SVM performed way better than applying PCA = 6. Actually in the validation phase no PCA performed slightly better so we assumed applying PCA = 6 would be more efficient but our assumption proved wrong. So the best model for polynomial SVM corresponds to having no PCA, $K = 0$ and $C = 10^{-2}$

4.2.2 RBF Kernel

We will also consider KernelRBF with PCA=6 and without PCA, taking into account $Z_{\text{norm}}=\text{true}$ and $\text{Rebal}=\text{false}$. We will fix $K=1$ and tune for different values of hyperparameter C and $\log \gamma$. Results are reported below:

	$C = 10^{-4}$	$C = 10^{-2}$	$C = 10^{-1}$	$C = 10^1$	$C = 10$
$\log \gamma = -2$	0.311	0.313	0.278	0.258	0.258
$\log \gamma = -3$	0.544	0.331	0.299	0.266	0.259
$\log \gamma = -4$	0.846	0.405	0.357	0.311	0.269

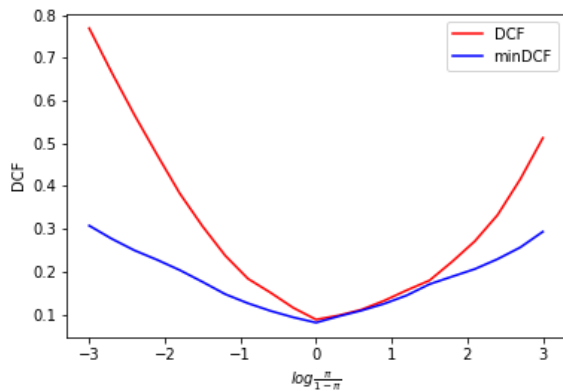
Table 28: RBF kernel results for PCA = 6, $Z_{\text{norm}}=\text{True}$, $\text{Rebal}=\text{false}$, $K=1$

	$C = 10^{-4}$	$C = 10^{-2}$	$C = 10^{-1}$	$C = 10^1$	$C = 10$
$\log \gamma = -2$	0.282	0.299	0.249	0.239	0.264
$\log \gamma = -3$	0.539	0.313	0.265	0.247	0.241
$\log \gamma = -4$	0.842	0.392	0.337	0.277	0.246

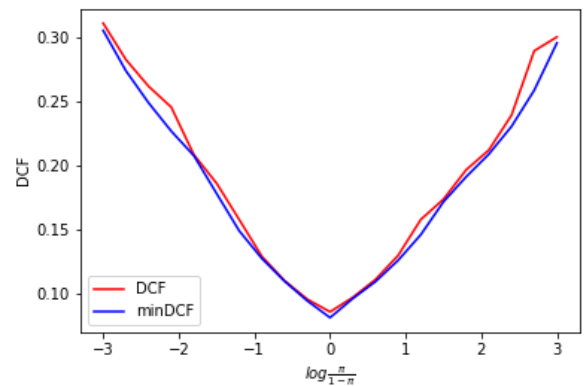
Table 29: RBF kernel results for no PCA, $Z_{\text{norm}}=\text{True}$, $\text{Rebal}=\text{false}$, $K=1$

We can see that KernelRBF with hyperparameters $C=10$ and $\log \gamma = -3$ with PCA=None, $Z_{\text{norm}}=\text{True}$, $\text{Rebal}=\text{false}$; performed much better than our chosen classifier on the evaluation data (that was PolyKernel SVM with $\text{minDCF}=0.269$ highlighted in blue above), again our choice was sub optimal.

Applying calibration to this model we get the following graphs, and we can see that with calibration the model performs significantly better:



(a) Non calibrated RBF SVM



(b) Calibrated RBF SVM

Figure 24: minDCF and DCF plots of best kernel RBF SVM

4.3 GMM

Finally, we consider the Gaussian Mixture Models. We focus on the best model Target Model=Diag (2 components) and NonTarget model = Diag (8 components) with PCA=None that resulted in minDCF= 0.218 on evaluation data set. But we will also try other combinations of components without and with PCA =6 since this dimensionality reduction proved to be efficient in previous models for comparison. The values of minDCF of those models are reported in the tables below:

Non-Target,Target	PCA 6	Non-Target,Target	no PCA
(4,2)	0.221	(4,2)	0.219
(8,2)	0.220	(8,2)	0.218

Table 30: *Results of Diag-Diag*

Non-Target,Target	PCA 6	Non-Target,Target	no PCA
(4,2)	0.221	(4,2)	0.223
(8,2)	0.217	(8,2)	0.216

Table 31: *Results of Diag-TiedDiag*

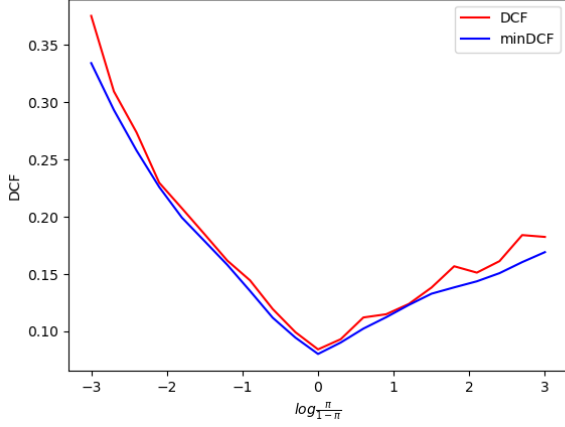
Non-Target,Target	PCA 6	Non-Target,Target	no PCA
(4,2)	0.253	(4,2)	0.230
(8,2)	0.223	(8,2)	0.214

Table 32: *Results of TiedDiag-Diag*

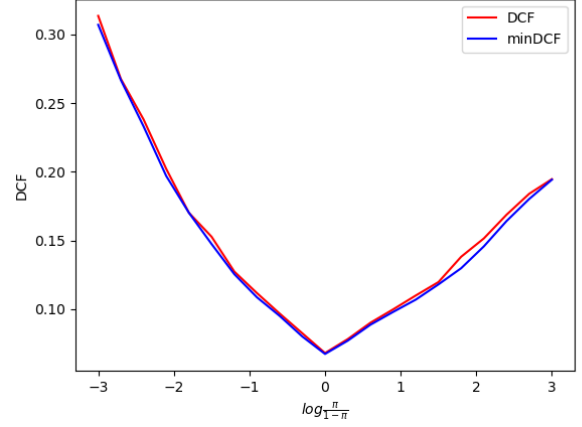
Non-Target,Target	PCA 6	Non-Target,Target	no PCA
(4,2)	0.254	(4,2)	0.234
(8,2)	0.222	(8,2)	0.214

Table 33: *Results of TiedDiag-TiedDiag*

Observing the above tables, we can see from the red highlighted values, that the models corresponding to non Target=Diagonal-Target=TiedDiag and non Target= TiedDiag and Target=Diag with (8,2) components and no PCA performed the best and are slightly better than our chosen model in validation phase ((8,2) Diag-Diag), but the difference is almost negligible. It's interesting to note that the model with PCA=6 and (Diag-TiedDiag) performed well and gave performance close enough to the one with no PCA so it might be a good idea to consider this model since it reduces dimensionality. We are going to apply calibration, to check how results may differ, we can see from the graphs below that GMM performed even better after calibration:



(a) Non calibrated best GMM model



(b) Calibrated best GMM model

Figure 25: *minDCF and DCF plots of best GMM model*

5 Conclusion

In the table below, we compare the minDCF values of our chosen models during validation for Quad LR, SVM, and GMM; with the best ones obtained with different model configuration during the evaluation phase:

Model	minDCF_val	minDCF_eval
GMM	0.218	0.214
Quad LR	0.251	0.247
SVM	0.269	0.241

Table 34

We observe that our decisions yielded satisfactory results for the Gaussian Mixture Model (GMM) and Quadratic Logistic Regression (Quadratic LR). Although certain alternative model configurations showed slightly better performance, they remained within a comparable range of outcomes as our selected models. However, in the case of Support Vector Machine (SVM), there is a notable difference in minDCF values, indicating that our choice of this classifier is sub-optimal. In summary, our overall conclusion is that, during the validation phase, the best model was the GMM. Furthermore, in the evaluation phase, it reaffirmed its status as the best performing model among the three considered options.